

LINUX ADMINISTRATION

LAB 2:

SSH + SFTP

FAIL2BAN

RSYSLOG



Prepared By: Hosni Zaaraoui

EPISODE1: SSH — SECURING REMOTE ACCESS

With our DHCP and DNS servers running, it's time to make sure we can access them securely. SSH is our new front door — but before we start using it, we'll change the locks and put in some real security.

Step 1 – Install and Enable SSH

On both SERVER-ALPHA and SERVER-BETA, install OpenSSH:

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh --now
```

```
hosniadmin@SERVER-ALPHA:~/Desktop$ sudo systemctl enable ssh --now
Synchronizing state of ssh.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
hosniadmin@SERVER-ALPHA:~/Desktop$
```

- ❖ `openssh-server` : provides SSH access.
- ❖ `systemctl enable ssh --now` : ensures SSH starts immediately and on every boot.!

Check that it's running:

```
sudo systemctl status ssh
```

Step 2 – Harden SSH Configuration

Open the SSH server configuration file:

```
sudo nano /etc/ssh/sshd_config
```

Apply the following changes:

```
Port 2222                # Move SSH from default port 22 to  
2222 (reduces random attacks)
```

```
PermitRootLogin no        # Block root login directly
```

```
PasswordAuthentication no # Disable password logins : enforce  
SSH keys only
```

```
AllowGroups sshusers      # Only members of 'sshusers' group  
can log in
```

Tip: In nano, press ALT+M to enable mouse support — it makes navigation much easier :)

In nano, save with CTRL+S, exit with CTRL+X.

Now create the allowed group and add your user:

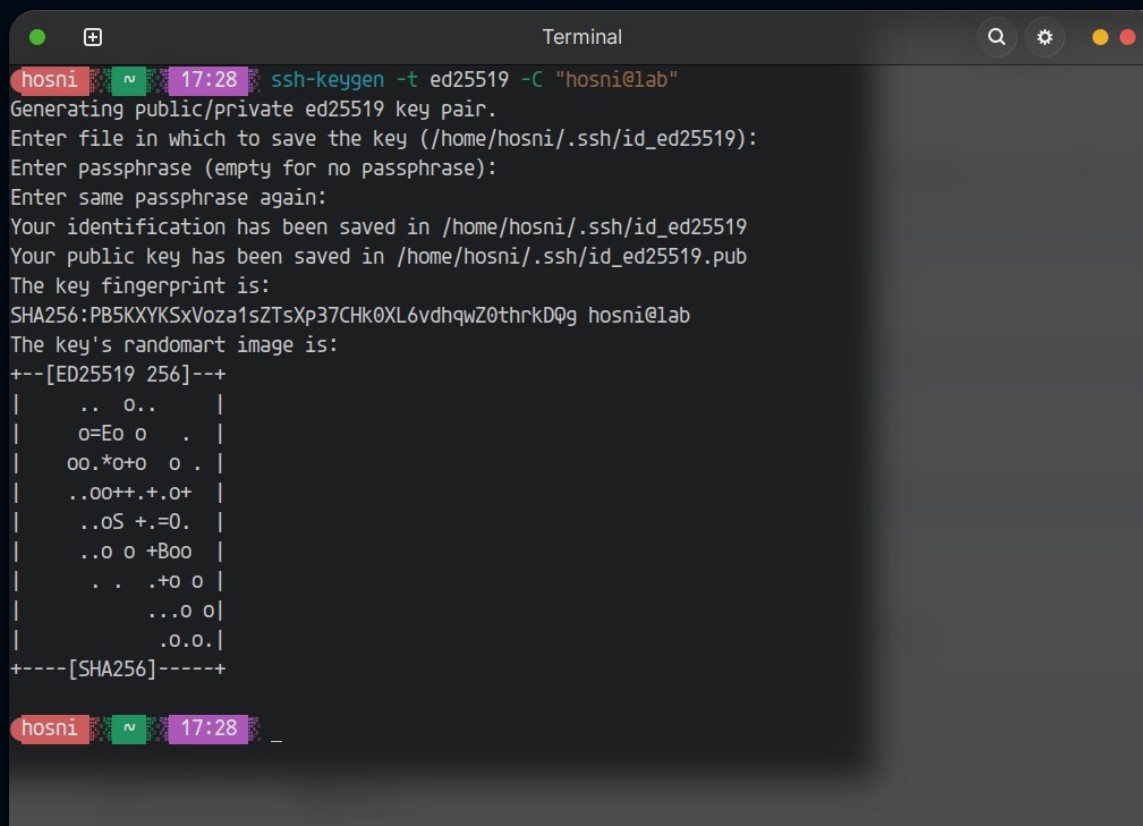
```
sudo groupadd sshusers
```

```
sudo usermod -aG sshusers hosni
```

Step 3 – Set Up SSH Keys

From your admin workstation (host machine or Kali), generate a key if you don't have one yet:

```
ssh-keygen -t ed25519 -C "hosni@lab"
```



```
Terminal
hosni ~ 17:28 ssh-keygen -t ed25519 -C "hosni@lab"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/hosni/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/hosni/.ssh/id_ed25519
Your public key has been saved in /home/hosni/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:PB5KXYKSxVoza1sZTsXp37CHk0XL6vdhqwZ0thrKDQg hosni@lab
The key's randomart image is:
+--[ED25519 256]--+
|    ..  o..   |
|    o=Eo o    |
|   oo.*o+o  o  |
|   ..00+++.+.o+ |
|   ..oS +.=0.  |
|   ..o o +Boo  |
|    . . .+o o  |
|           ...o o|
|           .o.o. |
+-----[SHA256]-----+
hosni ~ 17:28 _
```

Copy the public key to SERVER-ALPHA (adjust IP if needed):

```
ssh-copy-id -p 2222 hosni@192.168.50.10
```

```
Terminal
hosni ~ 19:05 ssh-copy-id -p 2222 hosniadmin@192.168.50.10
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 2 key(s) remain to be installed -- if you are prompted now it is to install all the new keys
hosniadmin@192.168.50.10's password:

Number of key(s) added: 2

Now try logging into the machine, with: "ssh -p '2222' 'hosniadmin@192.168.50.10'"
and check to make sure that only the key(s) you wanted were added.

hosni ~ 19:06 _
```

Test the connection:

```
ssh -p 2222 hosni@192.168.50.10
```

```
hosniadmin@SERVER-ALPHA: ~
hosni ~ 19:06 ssh -p 2222 hosniadmin@192.168.50.10
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.2.0-26-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Expanded Security Maintenance for Applications is not enabled.

426 updates can be applied immediately.
313 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

hosniadmin@SERVER-ALPHA:~$
```

If you log in without entering a password, **key authentication works**.

Step 4 – Configure SFTP with Chroot Jail

SSH also provides SFTP for secure file transfers. Let's set up a restricted SFTP user who can only access their own directory.

Edit SSH config again:

```
sudo nano /etc/ssh/sshd_config
```

Add this block at the bottom:

```
Match Group sftpusers
```

```
    ChrootDirectory /sftp/%u # Restrict users to /sftp/username
    ForceCommand internal-sftp # Force SFTP only (no SSH shell)
    AllowTcpForwarding no      # Disable tunneling
    X11Forwarding no           # Disable X11 forwarding
```

◆ AllowTcpForwarding no

Normally, SSH can be used not just to log in, but also to forward TCP connections (like tunneling traffic for other apps through SSH).

Example: a user could forward a local port and secretly access internal services (like a database or web app) that you didn't intend to expose.

Setting this to no means:

The SFTP-only user can't use SSH as a proxy or tunnel.

They're limited to file transfers, nothing more.

◆ X11Forwarding no

SSH has a feature called X11 forwarding, which lets users run graphical apps remotely (like opening Firefox or a GUI editor on the server and having it appear on your desktop).

For an SFTP-only user, this makes no sense and just increases attack surface.

Setting this to no means:

The user cannot start GUI applications over SSH.

It reduces risk from X11-based exploits.

Combined with ForceCommand internal-sftp, these two lines lock the user down to SFTP only — they can't use SSH tunnels, can't launch apps, can't break out.

Now create the SFTP group and user:

```
sudo groupadd sftpusers
```

```
sudo useradd -g sftpusers -d /incoming -s /sbin/nologin  
sftpuser
```

```
sudo passwd sftpuser
```

Prepare the chroot jail:

```
sudo mkdir -p /sftp/sftpuser/incoming
```

```
sudo chown root:root /sftp/sftpuser
```

```
sudo chown sftpuser:sftpusers /sftp/sftpuser/incoming
```

Restart SSH:

```
sudo systemctl restart ssh
```

Step 5 – Testing

Test SSH key login as your main user:

```
ssh -p 2222 hosni@192.168.50.10
```

Test SFTP login as restricted user:

```
sftp -P 2222 sftpuser@192.168.50.10
```

Inside SFTP, run:

```
pwd          # Should show /incoming
```

```
ls           # Should be empty unless you've added files
```

Success means your **SSH is hardened**, and **SFTP is restricted to its jail**.

EPISODE 2 : FAIL2BAN

Even with SSH hardened, attackers will still try their luck. That's where Fail2Ban comes in.

It monitors log files (like /var/log/auth.log) and bans IPs that show malicious behavior — usually by adding firewall rules.

Think of it as your 24/7 security guard: if someone keeps trying to break in, they get thrown out.

Step 1. Install and Enable Fail2Ban

On both servers:

```
sudo apt install fail2ban -y
```

```
sudo systemctl enable fail2ban --now
```

- ❖ fail2ban : the package itself.
- ❖ enable --now : runs it immediately and on every boot.

Step 2 – Configure Fail2Ban

By default, Fail2Ban watches for SSH on port 22. But since we changed SSH to port 2222, we need to adjust it.

Rule of thumb: never edit jail.conf directly. Instead, copy it to jail.local:

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

Find the [sshd] section and modify it like this:

```
[sshd]
enabled = true           # Enable SSH protection
port    = 2222           # Match our custom SSH port
filter  = sshd           # Use built-in SSH filter
logpath = %(sshd_log)s   # Where to watch login attempts
maxretry = 3             # Allow 3 failures
bantime = 3600           # Ban for 1 hour (3600s)
findtime = 600           # 3 failures within 10 minutes trigger a ban
```

Explanation:

- **maxretry = 3** : 3 bad logins allowed.
- **findtime = 600** : within 10 minutes.
- **bantime = 3600** : banned for 1 hour.

Step 3 – Restart and Test

Restart Fail2Ban:

```
sudo systemctl restart fail2ban
```

Check its status:

```
sudo fail2ban-client status sshd
```

```
Sorry but the jail sshd does not exist
hosniadmin@SERVER-ALPHA:~/Desktop$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:    0
| `-- File list:      /var/log/auth.log
`- Actions
   |- Currently banned: 0
   |- Total banned:    0
   `-- Banned IP list:
hosniadmin@SERVER-ALPHA:~/Desktop$
```

Now simulate a failed login attempt (from another machine or VM) with wrong key/password.

After 3 tries, the attacker's IP will be banned automatically.

```
Sorry but the jail: sshd does not exist
hosniadmin@SERVER-ALPHA:~/Desktop$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 1
| |- Total failed: 2
| `-- File list: /var/log/auth.log
`- Actions
  |- Currently banned: 0
  |- Total banned: 0
  `-- Banned IP list:
hosniadmin@SERVER-ALPHA:~/Desktop$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 3
| `-- File list: /var/log/auth.log
`- Actions
  |- Currently banned: 1
  |- Total banned: 1
  `-- Banned IP list: 192.168.50.1
hosniadmin@SERVER-ALPHA:~/Desktop$
```

With this, brute force bots waste their time.

EPISODE 3: RSYSLOG

Right now, logs are scattered across servers. That's fine for one box, but in HA setups, you want centralized logging.

That way, if SERVER-BETA fails or gets hacked, you still have its logs safely stored on SERVER-ALPHA.

We'll make SERVER-ALPHA (192.168.50.10) the log receiver, and SERVER-BETA (192.168.50.11) the log sender.

Step 1 – Configure the Receiver (SERVER-ALPHA)

Edit the rsyslog configuration:

```
sudo nano /etc/rsyslog.conf
```

Uncomment these lines to enable TCP reception:

```
module(load="imtcp")          # [Line 21] Load TCP input module  
input(type="imtcp" port="514") # [Line 22] Listen on TCP port  
514 for logs
```

Now, add a template file `/etc/rsyslog.d/remote.conf` to organize logs by host and date:

```
$template RemoteHost, "/var/log/remote/%HOSTNAME%/_%$YEAR%  
-%$MONTH%-%$DAY%.log"  
  
if ($fromhost-ip != '127.0.0.1' and $fromhost-ip !=  
'localhost') then ?RemoteHost  
  
& stop
```

Explanation:

- ❖ Logs from remote hosts are stored in
 /var/log/remote/<hostname>/<date>.log.
- ❖ & stop means: don't process them twice.

Restart rsyslog:

```
sudo systemctl restart rsyslog
```

Step 2 – Configure the Sender (SERVER-BETA)

Edit rsyslog config:

```
sudo nano /etc/rsyslog.conf
```

Add at the very bottom:

```
*.* @@192.168.50.10:514
```

- `*.*` : send all logs.
- `@@` : use TCP (one @ would mean UDP).
- `192.168.50.10:514` : send everything to SERVER-ALPHA.

Restart:

```
sudo systemctl restart rsyslog
```

Step 3 – Test Centralized Logging

On SERVER-BETA:

```
logger "LAB 2 SUCCESS: This message is being sent to SERVER-ALPHA!"
```

On SERVER-ALPHA, check the log file:

```
sudo tail -f /var/log/remote/SERVER-BETA/$(date +%Y-%m-%d).log
```

```
hosniadmin@SERVER-ALPHA:~$ sudo tail -f /var/log/remote/SERVER-BETA/2025-09-24.log
Sep 24 09:36:20 SERVER-BETA sudo: pam_unix(sudo:session): session closed for user root
Sep 24 09:36:20 SERVER-BETA rsyslogd: omfwd: remote server at 192.168.50.10:514 seems to have c
m system like a load balancer or firewall) shuts down or aborts a connection. Rsyslog will re-o
hich usually goes along with that behaviour). [v8.2112.0 try https://www.rsyslog.com/e/2027 ]
Sep 24 09:36:20 SERVER-BETA rsyslogd: action 'action-7-builtin:omfwd' suspended (module 'builti
reason for suspension. [v8.2112.0 try https://www.rsyslog.com/e/2007 ]
Sep 24 09:36:20 SERVER-BETA rsyslogd: action 'action-7-builtin:omfwd' resumed (module 'builtin:
Sep 24 09:36:20 SERVER-BETA systemd[1]: Starting Cleanup of Temporary Directories...
Sep 24 09:36:20 SERVER-BETA systemd[1]: systemd-tmpfiles-clean.service: Deactivated successfull
Sep 24 09:36:20 SERVER-BETA systemd[1]: Finished Cleanup of Temporary Directories.
Sep 24 09:36:23 SERVER-BETA hosniadmin: LAB 2 SUCCESS: This message is being sent to SERVER-ALPHA
Sep 24 09:37:13 SERVER-BETA rsyslogd[5061]: network unreachable resolving host/NS/TN1: 192.50.128.5
```

If you see the message, centralized logging is working.

If you haven't checked out Lab 1 (DHCP + DNS HA setup), you can find it here:

<https://www.linkedin.com/feed/update/urn:li:activity:7362071270168948736/>

